

Customer Segmentation with K-Means: Step-by-Step EDA Workflow

Executive Summary: We perform customer segmentation using two dataset options: (A) the **Mall Customers** dataset (with columns *CustomerID*, *Gender*, *Age*, *Annual Income (k\$)*, *Spending Score*) and (B) a **sample e-commerce transactions** dataset (with assumed *InvoiceNo*, *CustomerID*, *InvoiceDate*, *Quantity*, *UnitPrice*). For (B) we derive Recency, Frequency, Monetary (RFM) features to capture purchasing behavior [32†L139-L143] . The goal is to cluster customers into distinct groups with K-Means (a common unsupervised algorithm [34†L30-L37]), then profile each cluster for marketing. We walk through loading data, cleaning, EDA (including univariate/bivariate plots), feature scaling, choosing *K* (elbow & silhouette methods [34†L63-L66] [36†L413-L422]), applying K-Means, and interpreting clusters. Finally, we outline a sample dashboard (e.g. in Streamlit) to display the segments. Throughout we include runnable Python code (Pandas, NumPy, scikit-learn, Matplotlib/Seaborn) with brief comments. Key plots (histogram, boxplot, scatter, heatmap) are shown as examples.

Data Sources and Comparison

Dataset	Description	Pros	Cons
Mall Customers (A)	Demographics of 200 customers (age, gender, income, spending score). <i>Source: Kaggle Tutorial Dataset [28†L3-L11] .</i>	Very clean and simple; ready-made numeric features (Age, Income, Spending Score); ideal for illustrating clusters.	Synthetic learning dataset; may not reflect real purchase patterns. Limited features (no purchase history).
E-commerce Transactions (B)	Retail transaction log with invoices, dates, quantities, unit prices (e.g. UCI <i>Online Retail</i> dataset [8†L17-L25]).	Realistic, large-scale data; enables RFM feature engineering (Recency/Frequency/Monetary) [32†L139-L143] . Captures purchasing behavior.	Requires much cleaning (cancellations, missing IDs); larger size. More complex to aggregate into features.

Table: Pros/Cons of each dataset (Mall dataset is from Kaggle [28†L3-L11] ; E-commerce from UCI [8†L17-L25]).

1. Data Loading and Inspection

```
import pandas as pd
import numpy as np
# Option A: Mall Customers
```

```
mall_df = pd.read_csv("Mall_Customers.csv") # or use raw GitHub gist link
print(mall_df.shape)
print(mall_df.columns.tolist())
print(mall_df.head(3))
```

- For Mall dataset, we load the CSV (200 rows, 5 columns). Typical output (head) shows *CustomerID*, *Gender*, *Age*, *Annual Income (k\$)*, *Spending Score (1-100)*.
- For transactions (option B), we simulate a sample DataFrame:

```
# Option B: E-commerce transactions (synthetic example)
data = {
    'InvoiceNo': [1001,1002,1003,1004,1005,1006],
    'CustomerID': [1,2,1,3,2,3],
    'InvoiceDate':
    ['2021-12-01', '2021-12-03', '2022-01-15', '2022-01-20', '2022-01-25', '2022-02-01'],
    'Quantity': [2, 1, 1, 3, 2, 5],
    'UnitPrice': [50.0, 75.0, 60.0, 20.0, 80.0, 30.0]
}
ecom_df = pd.DataFrame(data)
ecom_df['InvoiceDate'] = pd.to_datetime(ecom_df['InvoiceDate'])
print(ecom_df)
```

- Check shapes, types (`.info()`), missing values (`.isna().sum()`), and duplicates (`.duplicated().sum()`). In the Mall dataset there should be no missing values. In a real retail set, one might drop rows with missing CustomerID or mark cancels (InvoiceNo starting with 'C').

2. Data Cleaning

```
# Mall dataset cleanup
mall_df = mall_df.drop_duplicates()
# Example: no missing in Mall, but if any nulls:
mall_df = mall_df.dropna()

# E-commerce cleaning (RFM prep)
ecom_df = ecom_df.drop_duplicates()
# Remove cancellations (if InvoiceNo starts with 'C', e.g. 'C1002')
ecom_df = ecom_df[~ecom_df['InvoiceNo'].astype(str).str.startswith('C')]
# Ensure numeric types
```

```
ecom_df['Quantity'] = pd.to_numeric(ecom_df['Quantity'], errors='coerce')
ecom_df['UnitPrice'] = pd.to_numeric(ecom_df['UnitPrice'], errors='coerce')
```

- We drop duplicates and NA rows. Mall data usually needs no cleaning. For transactions, we remove return records (those with InvoiceNo like "C1234"). We also convert types as needed. No severe outliers yet.

3. Feature Selection / Engineering

- **Mall:** We choose numeric features: *Age*, *Annual Income (k\$)*, *Spending Score (1-100)*. Gender could be used categorically, but K-Means needs numeric, so we focus on numeric features.
- **E-commerce (RFM):** We create **Recency**, **Frequency**, **Monetary** features per customer [32+L139-L143] . First, calculate each:

```
# Create a snapshot date (one day after last invoice)
snapshot_date = ecom_df['InvoiceDate'].max() + pd.Timedelta(days=1)
# Aggregate by customer
rfm = ecom_df.groupby('CustomerID').agg({
    'InvoiceDate': lambda x: (snapshot_date - x.max()).days,
    'InvoiceNo': 'nunique',    # frequency = number of invoices
    'UnitPrice': lambda x: np.sum(x * ecom_df.loc[x.index, 'Quantity'])
})
rfm.rename(columns={'InvoiceDate': 'Recency', 'InvoiceNo': 'Frequency',
0: 'Monetary'}, inplace=True)
rfm['Monetary'] = rfm['UnitPrice']
rfm = rfm[['Recency', 'Frequency', 'Monetary']].reset_index()
print(rfm)
```

- **Recency** is days since last purchase (lower = recent). **Frequency** is count of invoices. **Monetary** is total spend. These three columns form the clustering features for RFM segmentation.

4. Univariate EDA

We examine each feature individually.

- **Mall (A):** Age, Income, Spending Score distributions.

```
import matplotlib.pyplot as plt
import seaborn as sns

# Age histogram
sns.histplot(mall_df['Age'], kde=True)
plt.title("Age Distribution"); plt.xlabel("Age"); plt.ylabel("Count")
plt.show()
```

```
# Income boxplot
sns.boxplot(x=mall_df['Annual Income (k$)'])
plt.title("Annual Income (k$) Boxplot")
plt.show()

# Spending Score histogram
sns.histplot(mall_df['Spending Score (1-100)'], bins=10, kde=True)
plt.title("Spending Score Distribution"); plt.xlabel("Spending Score");
plt.ylabel("Count")
plt.show()
```

Example insight: The age distribution may be roughly uniform (18–70). Income often clusters (some may be outliers at high incomes). Spending Score ranges 1–100 roughly evenly. Boxplots help spot outliers (none extreme here).

- **E-commerce (RFM):** Check ranges of Recency, Frequency, Monetary:

```
sns.histplot(rfm['Recency'], bins=5)
plt.title("Recency Distribution (days since last purchase)")
plt.show()
sns.histplot(rfm['Frequency'], bins=5)
plt.title("Frequency Distribution (# of purchases)")
plt.show()
sns.histplot(rfm['Monetary'], bins=5)
plt.title("Monetary Distribution (total spend)")
plt.show()
```

This reveals if customers are recent vs dormant, how often they buy, and total spend distribution. (Typically, a few customers have very high monetary values, suggesting skew.)

5. Bivariate EDA

We explore pairwise relationships and correlation.

- **Mall (A):** Scatterplots of features:

```
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)',
data=mall_df, hue='Gender', alpha=0.7)
plt.title("Annual Income vs Spending Score")
plt.show()

sns.scatterplot(x='Age', y='Spending Score (1-100)', data=mall_df,
hue='Gender', alpha=0.7)
plt.title("Age vs Spending Score")
plt.show()
```

```
# Correlation heatmap
sns.heatmap(mall_df[['Age', 'Annual Income (k$)', 'Spending Score (1-100)']].corr(), annot=True)
plt.title("Correlation Matrix (Mall Data)")
plt.show()
```

The **Income vs Spending Score** scatter is key: clusters often emerge by eye (e.g. low income/low score, high income/high score, etc.). A sample scatter is shown below. The heatmap shows whether Age correlates with Income or Score (often weakly).

【45†embed_image】 *Figure: Example scatter of Annual Income vs Spending Score, colored by an attribute (e.g. gender). Clusters of customers (like high-income high-spend vs low-income low-spend) can be visually identified.*

- **E-commerce (RFM):** Pairwise plots:

```
sns.scatterplot(x='Recency', y='Frequency', data=rfm)
plt.title("Recency vs Frequency")
plt.show()

sns.scatterplot(x='Monetary', y='Frequency', data=rfm)
plt.title("Monetary vs Frequency")
plt.show()

sns.heatmap(rfm[['Recency', 'Frequency', 'Monetary']].corr(), annot=True)
plt.title("Correlation Matrix (RFM)")
plt.show()
```

Typically we see *Recency* and *Frequency* are inversely related (recent customers often buy more). The heatmap helps confirm: we might see a negative corr between Recency and Frequency, positive corr between Frequency and Monetary. This guides feature understanding before clustering.

6. Feature Scaling

Because K-Means uses Euclidean distance, **scaling features** is essential 【34†L51-L57】 . We apply StandardScaler (mean=0, unit variance) or MinMax.

```
from sklearn.preprocessing import StandardScaler

# Mall features (only Income and Score for clustering)
mall_features = mall_df[['Annual Income (k$)', 'Spending Score (1-100)']]
scaler = StandardScaler()
X_mall = scaler.fit_transform(mall_features)

# RFM features
```

```
rfm_features = rfm[['Recency', 'Frequency', 'Monetary']]
rfm_scaler = StandardScaler()
X_rfm = rfm_scaler.fit_transform(rfm_features)
```

Now all features are on comparable scales (e.g., mean ≈ 0). This prevents a large numeric range (e.g. Income) from dominating distance calculations [34†L51-L57] .

7. Choosing the Number of Clusters (K)

We use two methods: **Elbow Method** and **Silhouette Analysis** [34†L63-L66] [36†L413-L422] .

- *Elbow Method*: Compute total within-cluster sum of squares (WCSS) for $K=1..10$ and look for a “bend” (elbow) where improvement slows.

```
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

# Mall Elbow
wcsc = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_mall)
    wcsc.append(km.inertia_)
plt.plot(range(1, 11), wcsc, marker='o')
plt.title("Elbow Method (Mall Data)")
plt.xlabel("Number of clusters (k)")
plt.ylabel("WCSS")
plt.show()
```

- *Silhouette Score*: Measures how well samples are clustered (ranges from -1 to 1). Higher is better [36†L413-L422] . Compute for $K=2..10$:

```
from sklearn.metrics import silhouette_score

for k in range(2, 7):
    km = KMeans(n_clusters=k, random_state=42).fit(X_mall)
    score = silhouette_score(X_mall, km.labels_)
    print(f"k={k}, silhouette={score:.3f}")
```

[34†L63-L66] suggests that the elbow (or silhouette) helps choose a meaningful K . In practice for Mall data, both methods often indicate around **4-6 clusters**.

Repeat similarly for the RFM features:

```

# E-commerce Elbow
wcss = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X_rfm)
    wcss.append(km.inertia_)
plt.plot(range(1, 11), wcss, marker='o')
plt.title("Elbow Method (RFM Data)")
plt.xlabel("k")
plt.ylabel("WCSS")
plt.show()
# Silhouette for RFM
for k in range(2, 7):
    score = silhouette_score(X_rfm, KMeans(n_clusters=k,
    random_state=42).fit_predict(X_rfm))
    print(f"k={k}, silhouette={score:.3f}")

```

Suppose RFM elbow suggests **k=4** is reasonable.

8. K-Means Clustering and Results

We apply K-Means with the chosen K. For illustration, assume $k=5$ for Mall and $k=4$ for RFM.

```

# Mall K-Means (k=5)
kmeans_mall = KMeans(n_clusters=5, random_state=42)
mall_df['Cluster'] = kmeans_mall.fit_predict(X_mall)
print(mall_df.groupby('Cluster')[['Age', 'Annual Income (k$)', 'Spending Score
(1-100)']].mean())

```

This prints mean feature values per cluster, e.g. **Cluster 0**: (Avg age 29, Income 25k, Score 80) – “Young High Spenders”; **Cluster 1**: (Age 45, Income 80k, Score 35) – “High Income Low Spenders”, etc.

```

# RFM K-Means (k=4)
kmeans_rfm = KMeans(n_clusters=4, random_state=42)
rfm['Cluster'] = kmeans_rfm.fit_predict(X_rfm)
print(rfm.groupby('Cluster')[['Recency', 'Frequency', 'Monetary']].mean())

```

This shows e.g. **Cluster 0**: recent (low Recency), frequent, high spenders; **Cluster 1**: old (high Recency), low frequency/spend; etc.

9. Cluster Profiling and Marketing Insights

We interpret each cluster by its feature means:

- **Mall Data Example:**

- *Cluster 0 (e.g. Young-Low Income-High Spending):* Young customers with moderate income but high spending scores – possibly students or trend-seekers. *Marketing:* Trendy product ads, loyalty programs.
- *Cluster 1 (e.g. High Income-Low Spending):* Older, affluent but low spending – maybe occasional shoppers. *Marketing:* Upsell/premium offers or loyalty incentives to increase spend.
- *Cluster 2 (High Income-High Spending):* Affluent big spenders – likely premium customers. *Marketing:* Premium products, exclusive deals, VIP programs.
- *Cluster 3 (Low Income-Low Spending):* Price-sensitive, infrequent shoppers. *Marketing:* Discounts, budget-friendly promotions.
- *Cluster 4 (Middle segments):* Define similarly.

- **RFM (E-commerce) Example:**

- *Cluster 0 (Best Customers):* Very recent purchases, high frequency, high total spend. *Marketing:* Loyalty and retention (exclusive deals).
- *Cluster 1 (At-risk):* High recency (long inactive), low frequency, low spend. *Marketing:* Win-back campaigns, re-engagement offers.
- *Cluster 2 (Low-Value, New):* Recent but low spending/frequency. *Marketing:* Cross-sell to increase spend.
- *Cluster 3 (Loyal Moderate):* Medium recency/frequency/spend. *Marketing:* Regular updates, moderate offers.

Such profiling turns cluster metrics into **actionable recommendations** (for ads, promotions, retention) tailored to each group's behavior [32†L139-L143] .

10. Dashboard Sketch (Streamlit Example)

We can envision a Streamlit dashboard to present these segments:

```
# Example Streamlit code (not run here)
import streamlit as st
import matplotlib.pyplot as plt

st.title("Customer Segmentation Dashboard")
st.write("Clustering results and insights.")

# Show total customers and segment sizes
cluster_counts = mall_df['Cluster'].value_counts().sort_index()
st.bar_chart(cluster_counts)
```

```

# Scatter plot of clusters
fig, ax = plt.subplots()
sns.scatterplot(x='Annual Income (k$)', y='Spending Score (1-100)',
                hue='Cluster', data=mall_df, palette='tab10', ax=ax)
ax.set_title("Mall Customers Segments")
st.pyplot(fig)

# Example recommendations section
st.subheader("Marketing Strategy by Segment")
for i in sorted(mall_df['Cluster'].unique()):
    st.markdown(f"Cluster {i}: Describe strategy here...")

```

This skeleton demonstrates how one could embed cluster plots and stats in a web app. In a full project, we'd similarly visualize RFM clusters and allow interactive filtering.

11. Project Deliverables Checklist

- **Code:** Python scripts or notebooks for EDA, feature engineering, clustering (with imports as above).
- **Data Files:** `Mall_Customers.csv` ; `OnlineRetail.csv` (or synthetic data) [8†L17-L25] .
- **Plots:** Histograms of features, boxplots, scatter plots (e.g. Income vs Score [45†]), correlation heatmaps, elbow curves, silhouette plots. Include annotated examples in the report.
- **Results:** Tables of cluster profiles (mean values per segment) and recommended actions.
- **Dashboard:** Streamlit (or Plotly/Dash) app code showing segment counts and key visualizations.
- **Documentation:** This step-by-step report with code cells, comments, and references.

Each of the above items should be produced and clearly labeled in the final project.

Sources: We used the Mall Customers dataset (Kaggle learning dataset) [28†L3-L11] and the UCI Online Retail dataset [8†L17-L25] . We applied RFM concepts [32†L139-L143] and standard clustering techniques [34†L30-L37] [36†L413-L422] as cited above. The workflow is drawn from common data science practices in customer segmentation.